

Operating Systems

Assignment 4

Multithreading

Sambit Sahoo (22277)

*My username is **sambit** (You can find it in all screenshots.)*

Q1. a program to create a thread T1. The main process passes two numbers on T1. In result, T1 returns the sum to the parent process for printing.

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_4
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ vi a1.c
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ gcc a1.c -o a1
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ ./a1
Enter two numbers:
833
57
sum = 890
```

Program

```
#include<pthread.h>
#include<stdio.h>
#include<stdlib.h>

int sum;
void *add(void *arr){
    int *a = (int *)arr;
    sum = a[0] + a[1];
}

int main(){
    int arr[2];
    printf("Enter two numbers: \n");
    scanf("%d %d", &arr[0], &arr[1]);

    pthread_t tid;
    pthread_attr_t attr;

    pthread_attr_init(&attr); //default attributes
    pthread_create(&tid, &attr, add, (void *)arr);
    pthread_join(tid, NULL);
    printf("sum = %d\n", sum);
}

a1.c [+] 22,9 All
-- INSERT --
```

Explanation

- A thread is created using `pthread_create()` to perform addition.
- Two integers are taken as input from the user using `scanf()`.

- The input array is passed to the thread as a `void*` argument.
- Inside the thread function, the argument is typecast to `int*`.
- The thread computes the sum of the two numbers.
- A global variable `sum` is used to store the result.
- `pthread_join()` ensures the main thread waits for completion.
- The final result is printed in the main function.

Q2. a program that spawns two threads, T1 and T2. Thread T1 is responsible for generating a file named “data.txt,” and T2 is tasked with writing specific content to the “data.txt” file.

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_4
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ vi a2.c
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ gcc a2.c -o a2
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ ./a2
file created successfully.
Enter text to write:Sambit is writing the text for assignment4 question2.
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ cat data.txt
Sambit is writing the text for assignment4 question2.
to a file created by another thread.
```

Program

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<fcntl.h>
#include<unistd.h>
#include<string.h>

int fd;

//Thread 1 create file.
void *create_file(void *arg){
    fd = open("data.txt", O_CREAT | O_WRONLY, 0644);

    if (fd >=0) {
        printf("file created successfully.\n");
    }
    else{
        perror("file creation failed.");
    }

    pthread_exit(NULL);
}
```

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_4
}

//Thread 2 write content to it.
void *write_content(void *arg){
    char data[100];

    printf("Enter text to write:");
    fgets(data, sizeof(data), stdin);

    write(fd, data, strlen(data));
    close(fd);

    pthread_exit(NULL);
}

int main(){
    pthread_t t1, t2;

    // Create thread T1
    pthread_create(&t1, NULL, create_file, NULL);    //NULL when default attributes.
    pthread_join(t1, NULL); // ensure file is created first

    // Create thread T2
    pthread_create(&t2, NULL, write_content, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

Explanation

- Two threads are created using `pthread_create()` from the Pthreads library.
- Thread T1 creates the file `data.txt` using the `open()` system call.
- A global file descriptor `fd` is used to share access between threads.
- `pthread_join()` ensures that T1 completes before T2 starts execution.
- Thread T2 takes input from the user using `fgets()`.
- The input data is written to the file using the `write()` system call.
- The file is properly closed using `close()` after writing.
- Error handling is performed using `perror()` if file creation fails.

Q3. a program that spawns two threads, T1 and T2. Each of the thread accepts an 10,000 element array as an input, Thread T1 and T2 prints each and every array element by adding an integer 2 and multiplied by 4. Both the threads are canceled after 1 second from the main thread after spawning T1 and T2.

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_4
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ vi a3.c
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ gcc a3.c -o a3
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ ./a3 > data.txt
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ head data.txt
Threads starting...
a[0] = 3492
a[1] = 2636
a[2] = 2472
a[3] = 2868
a[4] = 3840
a[5] = 996
a[6] = 2124
a[7] = 3112
a[8] = 448
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$ tail data.txt
a[9991] = 920
a[9992] = 276
a[9993] = 400
a[9994] = 3392
a[9995] = 2876
a[9996] = 3488
a[9997] = 532
a[9998] = 1444
a[9999] = 956
Threads ending...
sambit@sambit-PC:~/Documents/Sem_8/OS/Assignment_4$
```

Program

```
sambit@sambit-PC: ~/Documents/Sem_8/OS/Assignment_4
#include<pthread.h>
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#include<unistd.h>

void *array_op(void *arg){
    int *arr = (int *)arg;
    for (int i = 0; i < 10000; i++){
        printf("a[%d] = %d \n", i, (arr[i] + 2) * 4);
    }
    return NULL;
}

int main(){
    pthread_t t1, t2;
    int array[10000];

    srand(time(NULL));
    for (int i = 0; i < 10000; i++){
        array[i] = rand() % 1000;
    }
    printf("Threads starting...\n");
    pthread_create(&t1, NULL, array_op, (void *)array);
    pthread_create(&t2, NULL, array_op, (void *)array);
    sleep(1);
    pthread_cancel(t1);
    pthread_cancel(t2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Threads ending...\n");
    return 0;
}
```

Explanation

- Two threads T1 and T2 are created using `pthread_create()`.
- An array of 10,000 elements is initialized with random values.
- The array is passed to both threads as a `void*` argument.
- Inside the thread function, the argument is typecast to `int*`.
- Each thread processes the array by computing $(arr[i] + 2) * 4$ for every element.
- Both threads execute concurrently and print the processed values.
- The main thread waits for 1 second using `sleep()`.
- Threads are terminated using `pthread_cancel()`.
- `pthread_join()` ensures proper cleanup after cancellation.